Computers and Operations Research 129 (2021) 105209

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/caor

An exact branch-and-price approach for the medical student scheduling problem

Babak Akbarzadeh *, Broos Maenhout

Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, 9000 Gent, Belgium

ARTICLE INFO

Article history:
Received 20 May 2020
Revised 6 November 2020
Accepted 2 January 2021
Available online 6 January 2021

Keywords:
Medical Student Scheduling problem
Branch-and-price
Two-stage pricing

ABSTRACT

In this paper, we consider the medical student scheduling problem, which is a tactical scheduling problem assigning medical students to specific disciplines and hospitals in order to ensure an appropriate training over a one-year scheduling horizon. These internship positions are offered by local hospitals that specify minimum and maximum staffing requirements. To some extent, students can customise their own training program as they need to select a subset of the disciplines from an elective list and they are able to express their preferences related to these disciplines and the hospitals in which they would like to carry out their training program. The objective is to find a feasible student roster satisfying all educational and hospital requirements while optimising the student preferences and the equity between students. We propose an efficient branch-and-price algorithm to solve the problem under study. To that purpose, we include different dedicated mechanisms to accelerate the algorithm performance to find the exact solution in an acceptable time span. In this way, we propose a two-stage pricing procedure finding a diverse set of suitable columns to reduce the tailing-off effect in column generation. The pricing algorithm relies on dynamic programming and different pruning and symmetry breaking rules. In addition, the branch-and-price tree is constructed based on a mixed-shifting branching rule that executes different individual branching rules in a specific order. The created nodes in the branching tree are visited in the order determined by a novel right-first search strategy. We conducted different computational experiments to show the performance of the proposed branch-and-price and the implemented speed-up techniques and optimisation principles. We have benchmarked the branching and pricing methodology with other approaches and show the contribution of the different components in the algorithm.

© 2021 Elsevier Ltd. All rights reserved.

1. Introduction

Upon graduation from medical school or university, students need to complete a clinical training to fulfil the specialty board certification requirements to work as an independent physician. This training implies that during different academic years students follow a one-year residency training program, organised by the medical school. Such a one-year training program, which is linked to the seniority level of the student, embodies a list of disciplines (e.g. critical care, gynaecology, general surgery) that should be carried out at local hospitals under the supervision of an attending physician. The available number of student positions offered by the hospitals are constrained and may vary over time. To fulfil the training program, students must attend all or a subset of these disciplines and practice these during a certain period. The disciplines are typically organised in a group of mandatory disciplines and a group of elective disciplines out of which the student can choose different disciplines. Moreover, students can indicate their availability and their preferences to carry out a specific discipline in a particular hospital. In this way, the students have the opportunity to focus on one or more (sub-) specialties according to their interests and the training program is customised to individual students. Incorporating these student preferences is highly important since the quality of the schedule has a significant impact on the students' life and education and on the delivered care to patients (Cohn et al., 2009; Rose et al., 2015). In addition, according to Colbert et al., 2017, not only the student preferences but also the fairness between the training programs of individual students need to be considered in order to safeguard the care quality delivery and effective learning.

In this study, we address the medical student scheduling problem, which is a tactical scheduling problem and assigns medical students to a (predefined) set of medical disciplines over a given time horizon in order to ensure the students receive an appropriate

* Corresponding author.
E-mail addresses: Babak.Akbarzadeh@Ugent.be (B. Akbarzadeh), Broos.Maenhout@Ugent.be (B. Maenhout).

https://doi.org/10.1016/j.cor.2021.105209
0305-0548/© 2021 Elsevier Ltd. All rights reserved.

medical training. The individual training schedules are constructed in line with the student preferences and the (hard) educational requirements stipulated by the medical school. These regulations define the duration of the training program and of the relevant individual disciplines, the required (number of) disciplines and the precedence relationships between different disciplines. The individual training schedules for all students build the student roster, which should satisfy the (hard) minimum and maximum student staffing requirements stipulated by the hospitals. The objective of the problem is to find equitably-efficient schedules, optimising on the one hand the total student desire and on the other hand the fairness across students as the quality of the worst student schedule is maximised. In this way, the problem considers the elementary conditions for defining a student scheduling problem, identified by Guo et al. (2016), enriched by other relevant constraints and objectives that are recognised as indispensable in academic literature and real life (Akbarzadeh and Maenhout, 2020).

In contrast to the related research of Akbarzadeh and Maenhout, 2020, who developed a heuristic procedure to solve large-scale real-life problems, the focus of this paper is to propose an efficient exact solution method that is able to solve problems of reasonable size in an exact manner and an acceptable time span. To that purpose, we decompose the original problem formulation using the Dantzig-Wolfe decomposition in a master problem relying on column variables, which represent feasible student schedules, and a subproblem employing the original decision variables to define feasible student schedules. The decomposed problem is solved using a dedicated branch-and-price procedure, which applies a column generation algorithm in every node of the branching tree to find the optimal LP solution and a branching method to drive the LP solution, when fractional, to integrality. In order to enhance the branch-and-price algorithm, we introduce several dedicated mechanisms, which improve the computational performance dramatically. First, to solve the pricing problem, the most time consuming component of the algorithm, we propose an efficient two-stage algorithm based on dynamic programming. The first stage aims to generate a diverse set of promising columns in a heuristic manner to stabilise the column generation algorithm and to reduce the tailing-off effect. The second stage aims to find the best column in an exact manner. Second, we implement different pruning rules and a symmetry breaking rule in order to speed up the performance of the dynamic programming algorithm. Third, in each node of the branching tree, we determine high-quality solutions quickly based on the optimal LP solution using a greedy heuristic and a Hungarian-based heuristic. Fourth, we customise the search for an integer solution based on the optimal LP relaxation and the imposed branching constraints. The exploration of the branching tree in search for the optimal integer solution relies on a mixed-shifting branching rule that alternates between different individual branching rules based on the allocated capacity and the original decision variables. The nodes in the branching tree are explored in the order defined by a novel right-first search strategy. In order to validate these optimisation principles in a computational manner, we conduct experiments on a synthetic dataset that is generated in a controlled manner, guided by the problem features encountered in real life and academic literature. We analyse the design choices of the proposed algorithm component by component and show how each component contributes to the performance of the algorithm, reducing the required computational effort and/or the optimality gap. In this analysis, the proposed algorithm is benchmarked with different other solution methodologies suggested by the literature.

The remainder of the paper is organised as follows. In Section 2, we discuss the relevant literature concerning the medical student scheduling problem and solution approaches. In Section 3, we describe the problem under study. Section 4 provides the proposed solution methodology. In Section 5, we discuss the computational performance of the algorithm. Section 6 provides concluding remarks and the contribution of the proposed research.

2. Related literature

The medical student scheduling problem has been studied in different forms for which different dedicated solution methodologies have been proposed in the literature.

2.1. Problem characteristics

The problem involves different stakeholders, i.e. medical schools, hospitals and students, for which different features and objectives have been defined. All studies include some educational requirements to ensure that students acquire the aspired medical proficiency, i.e. the students should carry out a particular (fixed) set of disciplines or activities (e.g. Smalley and Keskinocak, 2016; Bard et al., 2017; Proano and Agarwal, 2018; Diponegoro and Rukman, 2018). Only a few studies consider some specific but very relevant curriculum rules, complicating the problem significantly. Ito et al. (2017) included an elective set of medical disciplines such that students can choose and customise their program based on their interests. Ito et al. (2017) and Brech et al. (2019) argue for the necessity of (in) direct precedence relationships between disciplines (e.g., anaesthesiology is a predecessor for general surgery). Student staffing requirements, defining the capacity offered by hospitals, are regularly considered in the literature as hard constraints and occasionally as soft constraints (Bard et al., 2013; Bard et al., 2016; Bard et al., 2017). Nearly all studies consider a maximum student capacity (e.g. Smalley and Keskinocak, 2016; Proano and Agarwal, 2018). In some studies, minimum staffing requirements have been imposed since hospitals may count on students as a part of the required staff to cope with the patient workload (e.g. Beliën and Demeulemeester, 2006; Ryan et al., 2013; Guo et al., 2014). Student characteristics are regularly taken into account when assigning students to the available positions. The student availability over time is a common feature in many studies (e.g. Beliën and Demeulemeester, 2006; Proano and Agarwal, 2018). In addition, the optimisation of the student preferences has been the most popular objective in the literature. In this perspective, Smalley and Keskinocak (2016), Diponegoro and Rukman (2018) and Ito et al. (2017) amongst others have considered student preferences to carry out a particular discipline. Akbarzadeh and Maenhout (2020) include other types of student preferences such as hospital-related preferences, waiting times between discipline assignments and the changes of students between hospitals in consecutive periods, which may be necessary when the capacity of the involved hospitals is varying over time and/or the hospitals are different entities located in distant places. Schleyer (1994) and Akbarzadeh and Maenhout (2020) define a max–min fairness objective for the medical student scheduling problem to determine as best as possible a fair and equal training of students, which is important from the perspective of the medical school.

2.2. Solution methodology

Due to the size of the problems encountered in practice, finding a feasible student roster has shown to be a very difficult task. From a theoretical point-of-view, the feasibility problem related to a basic version of the medical student scheduling problem has been proven to be NP-complete (Guo et al., 2014). This basic version contains only the elementary constraints, i.e. (i) each student

requires a schedule allocating the student during each time period to exactly one discipline or idle period, (ii) the (hospital) capacity constraints per time period embodying a minimum and maximum student demand, and (iii) educational requirements specifying the minimum training requirements. The problem is further complicated empirically when optimising the roster in accordance to the student preferences, maintaining fairness among the students and/or considering additional constraints (e.g. precedence relations between disciplines) (Hurink and Knust, 2001). As a result, the majority of studies in the literature propose a (constructive) heuristic procedure relying upon some decomposition of the problem or the relaxation of some (hard) constraints to compose a student roster quickly. A repair mechanism or local search method is typically appended to restore the feasibility or to improve the solution locally. In this way, different authors (e.g. Schleyer, 1994; Guo et al., 2014; Diponegoro and Rukman, 2018) apply a student-based decomposition and schedule individual students one by one in order to improve the student schedule with the lowest number of disciplines assigned. Other authors use mathematical programming and solve first an easier problem relaxing some functional constraints or the integrality conditions and then use a heuristic procedure to repair the infeasibilities related to the relaxed constraints (Franz and Miller, 1993; Bard et al., 2013; Bard et al., 2016). Akbarzadeh and Maenhout (2020) use different decomposition-based heuristics to first construct a heuristic solution period by period and further improve the constructed solution locally student per student and position per position.

Only a few studies propose an exact solution methodology based on mathematical programming (Beliën and Demeulemeester, 2006; Beliën and Demeulemeester, 2007; Smalley and Keskinocak, 2016; Ito et al., 2017; Proano and Agarwal, 2018; Kraul et al., 2019). In order to solve the problem efficiently, Beliën and Demeulemeester (2006), Beliën and Demeulemeester (2007) and Kraul et al. (2019) reformulate the original problem into a set partitioning problem using the Dantzig-Wolfe decomposition. Column generation is applied to solve the LP relaxation of the reformulated model and either heuristic principles (e.g. Kraul et al., 2019) or an exact branching scheme are used to transform the optimal LP solution into an integer solution. These exact approaches proposed in literature are tested on smaller-sized test instances and/or have demonstrated to only slowly converge towards the optimal solution. In general, solution methodologies based on the Dantzig-Wolfe decomposition have been proven to be useful methods for solving large-scale optimisation problems, providing a tighter convex hull of the solution space. However, deriving an efficient column generation and/or related branch-and-price methodology is not straightforward as (i) the decomposition leads to an integer pricing subproblem, (ii) many column generation iterations may be required to prove LP optimality and (iii) branching on the so-called column variables of the master problem leads to an unbalanced branching tree and destroys the structure of the pricing problem (Barnhart et al., 1998). As a result, performance needs to be accelerated by devising efficient solution procedures for the pricing problems, preferably exploiting problem-specific knowledge, and dedicated branching schemes to limit the size of the branch-and-price tree, possibly relying on pruning rules and symmetry breaking rules.

To solve the pricing subproblem, Beliën and Demeulemeester (2006) and Beliën and Demeulemeester (2007) use a standard dynamic programming approach whereas (Kraul et al., 2019) employ a commercial MIP software package to solve the pricing problem using mathematical programming. Note that in other scheduling domains, constraint programming has been applied as well as there is no need to solve the pricing problem to optimality, defining a feasibility pricing problem (e.g. Junker et al., 1999; Pisinger and Sigurd, 2007; He and Qu, 2012; Doulabi et al., 2016). According to Morrison et al. (2016), the creation of nodes and the search method throughout the branching tree has a major influence on the performance of a solution methodology. The search strategy in a tree determines the order in which unexplored nodes (subproblems) are selected for exploration. Different search strategies have been introduced in the literature, i.e. best-node-first, deepest-node-first, breadth-first and a cyclic best-node-first search as a combination of the best-first and the deepest-first search. In a branch-and-price algorithm, an integer solution is expected in the early stages of the solution procedure as a consequence of the tight convex hull delivered in each node via column generation (Barnhart et al., 1998). As a result, most branch-and-price solution methodologies implement a best-first search as e.g. done by Beliën and Demeulemeester (2006), Beliën and Demeulemeester (2007) for the medical student scheduling problem. The branching strategy stipulates the method to create disjoint subproblems based upon the LP relaxation obtained in a node, which is performed via either a binary branching or a wide branching rule creating multiple subproblems (Morrison et al., 2016). Beliën and Demeulemeester (2006) examine different branching rules for the medical student scheduling problem to find the optimal integer solution, i.e. branching on the column variables of the master problem, branching on the original assignment variables of the pricing problem and a precedence-related branching scheme. They conclude that branching on the original assignment variables leads to the most balanced branching tree and the best computational performance.

3. Problem definition and formulation

The Medical Student Scheduling problem assigns graduate students to a specific set of medical disciplines over the course of the academic year to fulfil his/her clinical training. A training program is designed by a medical school or university and embodies a list of disciplines related to relevant medical specialties. In order to fulfil their individual training program, students are assigned to disciplines across different wards and hospitals, where they work during one or multiple (successive) periods under the supervision of an attending physician. Every ward and hospital defines a minimum and maximum number of student positions per time period. The curriculum manager is the administrative responsible, belonging to the medical school, overseeing the construction of the student trainee roster to ensure that training and learning needs are addressed in accordance to the requirements of the different stakeholders. The scheduling horizon is divided in multiple time periods, having a length of e.g. one month, and the students are assigned to disciplines period by period. Students are scheduled in accordance with (i) the stipulations of the medical school to set up a qualitative training schedule, (ii) the limited capacity of the involved wards and hospitals for accepting students and (iii) the availability indicated by the students. An individual student can customise his/her individual training schedule, further denoted as a *student schedule*, to some extent by electing some of the disciplines on top of a number of mandatory disciplines and the hospitals they wish to carry out the allocated disciplines. These elective disciplines are chosen from predefined lists, which are composed based upon the student track of study (e.g. general medicine, revalidation medicine) and seniority level, to ensure the students receive an appropriate medical training. The set of selected

individual schedules for all students constitute the so-called *student roster*. The objective is to set up a feasible equitably-efficient timetable maximising the total sum of desire scores and the quality of the worst schedule to ensure equity among students.

In Section 3.1, we describe the constraints and objectives of the Medical Student Scheduling problem under study. Section 3.2 gives an overview of the main decision variables of the problem under study and provides the compact mathematical formulation of the problem. The problem under study originates from the observation and analysis of a real-life setting encountered in the Faculty of Medicine and Health Sciences (Ghent University, Belgium), which has been investigated by Akbarzadeh and Maenhout (2020). However, we consider only the elementary features of a student scheduling problem, identified by Guo et al. (2016), appended with the most prominent objectives and relevant other constraints that are indispensable in constructing a suitable student schedule for a real-life problem. The problem under study is relevant for the scheduling of students involving different types of study programmes, characterised by a broad range of valid problem sizes (see Akbarzadeh and Maenhout, 2020 for an overview). We employ the following notation of the involved sets and parameters to describe the problem under study:

Notation	
Sets	
S	The set of students (index s and $\bar{s}$)
D	The set of disciplines (index d and $\bar{d}$)
G	The set of discipline groups (index g)
T	The set of time periods in the scheduling horizon (index t)
W	The set of wards (index w)
H	The set of hospitals (index h)
Parameters	
Medical school	
κ_{sg}	The total number of disciplines student s must attend from discipline group g
φ_{sdg}	1, if discipline d is included in discipline group g for student s; 0, otherwise
Du_d	Duration of discipline d
$Sk_{\bar{d}d}$	1, if the precedence relationship $\left(\bar{d}, d\right)$ is defined, i.e. discipline $\bar{d}$ has to be carried out before discipline d; 0, otherwise
Λ	The maximum number of disciplines that students can fulfil in a particular hospital
Hospitals	
ξ_{dwh}	1, if discipline d can be carried out in ward w, hospital h; 0, otherwise
Dem^{Min}_{wht}	Minimum student requirements for ward w, hospital h, period t
Dem^{Max}_{wht}	Maximum student requirements for ward w, hospital h, period t
Medical students	
Av_{st}	1, if student s is available during time period t; 0, otherwise
Ab_{sdh}	1, if student s has the ability to carry out discipline d in hospital h; 0, otherwise
Pr^{disc}_{sd}	Preference for student s to carry out discipline d
Pr^{hosp}_{sh}	Preference for student s to carry out a training in hospital h
Pr^{trPr}_{d}	General preference to carry out discipline d
BM	Big number
Preference weight	
ω^{disc}_{s}	Weight related to the discipline preferences of student s
ω^{hosp}_{s}	Weight related to the hospital preferences of student s
ω^{trPr}	Weight related to the general discipline preferences in the training program
ω^{chng}_{s}	Weight related to the changes between hospitals for student s
ω^{wait}_{s}	Weight related to the waiting times for student s
Objective function weights	
α	Objective weight related to maximising the desire score for all students
β	Objective weight related to maximising the fairness between students

3.1. Description of the objectives and constraints

The problem encompasses three main stakeholders, i.e. the medical school, the local hospitals and the medical students, each having their requirements and objectives. The curriculum manager has to construct a roster assigning all students to different disciplines offered by different hospitals over the scheduling horizon while considering the student characteristics and the educational requirements of the training program set by the medical school. In the following, we discuss the constraints imposed on the composition of an individual student schedule (Section 3.1.1) and the roster for all students (Section 3.1.5). In Section 3.1.6, we discuss the considered objective components.

3.1.1. The rules and regulations imposed on an individual schedule

A student schedule has to be composed for every individual student respecting the following requirements, i.e.

Medical school

The medical school sets *educational requirements* as it is responsible for the quality of the training program in order to ensure that each student from the involved set of students S acquires the intended final competences. A training program involves a set of disciplines D, which are categorised into a set of groups G by the curriculum manager based upon e.g. the importance of a discipline and/or the discipline type. As a result, different groups or lists are constructed, which contain particular disciplines (indicated by φ_{sgd}). From each of the defined groups, a student has to carry out a minimum number of disciplines κ_{sg} during the scheduling horizon, consisting of a set of time periods T. The notion of groups is typically used to define one group comprising the mandatory disciplines that have to be carried out by the student, and one or multiple groups of elective disciplines that can be assigned to the student based on his preferences. E.g. a student has to carry out nine disciplines, of which seven disciplines are mandatory (General medicine, Surgery, Internal Medicine, Paediatrics, Psychiatry, Rheumatology, Emergency medicine), two disciplines are chosen out of the elective group (Movement therapy, Physiotherapy, Orthopaedics, Gynaecology, Neurology, Otolaryngology, Dermatology). Note that although it is possible that a particular discipline, a so-called mutual discipline, is included in different groups for a particular student, the total number of disciplines that the student must fulfil in the training program is predefined and equal to $\sum_g \kappa_{sg}$. A discipline is performed by a student during a certain period with length Du_d, depending upon the specific discipline.

Further, an (indirect) precedence relationship may be defined between certain pairs of disciplines, i.e. a specific discipline d can only be performed if the student has acquired the competency on another discipline $\bar{d}$ in an earlier period, denoted by $Sk_{\bar{d}d}$. Moreover, a student can only carry out a limited number of disciplines in a single hospital, indicated by the parameter Λ. The latter rule has been stipulated by the curriculum manager to ensure students acquire experience in different working atmospheres and to improve the fairness in the student-hospital assignments from the perspective of both the students and hospitals.

Hospital

The internships are offered by a set of local hospitals H, such that the student carries out the training program at distant locations. Each hospital is composed of a set of wards W referring to the physical place (or building), where the student can carry out a specific set of disciplines. A ward is the smallest entity that offers a number of unique student positions involving one single discipline or a set of related disciplines between which the resources are shared (e.g. 'Basic gynaecology' and 'Specialised gynaecology'). Note that if a single department in the hospital has distinct, unique positions for two (related) disciplines, the student staffing requirements are modelled by defining two ward entities for each one of these disciplines. A ward and hospital can set *(hospital) requirements* to select a student (e.g. a gender-specific rule accepting only female students, the physical ability of a student), which is referred to as the student-ward ability, indicated by Ab_{sdh}.

Medical student

A student may set his/her own *student-specific requirements*. In this regard, a student can only carry out a discipline during a period if (s) he is available in that period, indicated by the parameter Av_{st}.

3.1.2. The rules and regulations for a feasible student roster

In addition to the constraints imposed on the individual student schedules, hospital resource capacity constraints, defined on the level of the ward entity, are imposed on the construction of a roster for all students. For each ward and hospital, a minimum and maximum student requirement is imposed, which may be different from period to period, denoted by Dem^{Min}_{wht} and Dem^{Max}_{wht} respectively.

3.1.3. Objective function

Apart from the multitude of (hard) constraints, which define a feasible roster, we consider the following two objectives related to the preferences of the students, i.e.

(1) **Maximise the student desire**: This objective optimises the quality of the schedule by maximising the total sum of desire scores, which considers the student perspective. In calculating the desire related to the schedules of the students, we take into account (i) the discipline preferences of the students and (ii) the hospital preferences defined by the students; (iii) a more general discipline preference for all students defined by the curriculum manager, who may want to steer students towards specific disciplines depending on their training program (for reasons of seniority, soft precedence relations between disciplines, limited hospital capacity, etc), certainly if students are indifferent towards the different elective disciplines; (iv) the number of changes for students between hospitals in order to minimise the administrative effort and the physical changes for the students; and (iv) the waiting times between consecutive discipline assignments.

(2) **Maximise the fairness between students**: The second objective considers the perspective of the curriculum manager and aims to optimise the fairness between the students' schedules. The literature has proposed various inequity-averse optimisation criteria that can be used to model distribution equity between different agents alongside the efficiency criterion (see Karsu and Morton, 2015 for an overview). Approaches may be focusing either on the max–min fairness, maximising the worst schedule outcome across agents following the Rawlsian approach (e.g. Ronnberg and Larsson, 2010; Nishi and Izuno, 2014), or may embody inequality index-based approaches taking into account the degree of inequality between agents (e.g. minimise the range between the minimum and maximum solution quality realised across the agents (Trilling et al., 2006; Stolletz and Brunner, 2012)), which requires a significant higher computational effort. In this paper, this fairness objective is implemented via a max–min objective, i.e. we maximise the worst or minimum desire score (Des^{min}) attained by one of the students. This criterion is based on the Rawlsian principle (Rawls, 1971) and is widespread in the academic literature, providing a large equity between agents since the constraints imposed on the schedules of different students are very similar.

These two objectives are aggregated and formulated as a weighted sum with appropriate weights since the different objective function components are interrelated. Hence, the objective function is to maximise the weighted sum of the student desire (efficiency criterion) and to maximise the minimum desire (equity criterion). Karsu and Morton (2015) discuss that such an aggregated approach that incorporates a fairness criterion on top of the efficiency criterion, encourages an equitable distribution between agents. In this way, we aim to generate equitably-efficient timetables, i.e. we cannot improve the worst student schedule at the expense of a schedule of another student with a higher quality without deteriorating the overall schedule quality.

3.2. Compact problem formulation

The mathematical formulation of the Medical Student Scheduling problem embodies a multi-objective mixed integer programming (MIP) model. The compact formulation of this problem relies on two main decision variables, i.e. (i) the assignment variables v_{sdth}, which indicate the allocation of student s to discipline d, time period t and hospital h; and (ii) a network flow decision variable $y_{s\bar{d}dh}$, which indicates that discipline d in hospital h is assigned to student s after completing discipline $\bar{d}$. These two variables are referred to as the original decision variables to model the problem under study. The start time of a discipline assignment d is calculated as the start time and duration of the preceding discipline $\bar{d}$ plus the waiting time before discipline d. The network flow variables are used in a multi-commodity network flow model, which consists of a set of nodes representing the disciplines to which the students can be assigned and a dummy start node initiating all flow. The variables $y_{s\bar{d}dh}$ represent the flow over the arcs in the network flow model between two disciplines that are carried out in successive order.

Decision variables	
v_{sdth}	1, if student s starts discipline d in hospital h, time period t; 0, otherwise
$y_{s\bar{d}dh}$	1, if student s follows discipline d in hospital h directly after discipline $\bar{d}$; 0, otherwise
Des_s	The aggregated desire score for student s
Des^{min}	The worst desire outcome obtained across students
W_{sd}	The number of time periods student s has to wait before starting discipline d
Ch_{sd}	1, if student s changes from hospital to carry out discipline d; 0, otherwise

$$\text{Maximize } Z = \alpha \sum_{s \in S} Des_s + \beta Des^{min} \tag{1}$$

The objective function (1) embodies a weighted function maximising the total sum of the student desire scores and the worst or minimum desire score across students. Given predefined weight settings for α and β, a Pareto optimal solution is generated maximising both the efficiency and equity across students. This objective is subject to the following constraints, i.e.

Educational requirements set by the medical school

Constraint (2) stipulates the required number of disciplines that a student should select from each discipline group. Eq. (3) stipulates the total number of courses a student must attend, which is equal to the sum of disciplines required to be taken from all groups as the student may carry out a so-called mutual discipline that can be elected from multiple discipline groups. Constraint (4) designates that each discipline can be directly preceded by at most one other discipline. Eq. (5) specifies that in the network flow model the flow for each student is originated in the dummy start node, indicated by discipline 0. Eq. (6) indicates that a particular discipline cannot be succeeded by the same discipline or the dummy start node. Constraint (7) ensures a feasible flow between disciplines, imposing that starting from the preceding discipline $\bar{d}$, discipline d can only be selected if there is also a preceding discipline $\bar{\bar{d}}$ for $\bar{d}$. Eq. (8) indicates that a discipline can only start once when selected. Constraint (9) ensures the required precedence between particular pairs of disciplines, imposing that the preceding discipline $\bar{d}$ must be carried out prior to the particular discipline d. Constraint (10) imposes the maximum number of disciplines that can be carried out in a particular hospital by a student.

$$\sum_{d \in D} \varphi_{sgd} \sum_{t \in T} \sum_{h \in H} v_{sdth} \geqslant \kappa_{sg} \qquad \forall s \in S, \forall g \in G \tag{2}$$

$$\sum_{d \in D} \sum_{t \in T} \sum_{h \in H} v_{sdth} = \sum_{g \in G} \kappa_{sg} \qquad \forall s \in S \tag{3}$$

$$\sum_{\bar{d} \in D} \sum_{h \in H} y_{s\bar{d}dh} \leqslant 1 \qquad \forall s \in S, \forall d \in D \setminus \{0\} \tag{4}$$

$$\sum_{d \in D} \sum_{h \in H} y_{s0dh} = 1 \qquad \forall s \in S \tag{5}$$

$$\sum_{s \in S} \sum_{d \in D} \sum_{h \in H} (y_{sddh} + y_{sd0h}) = 0 \tag{6}$$

$$\sum_{d \in D} \sum_{h \in H} y_{s\bar{d}dh} \leqslant \sum_{\bar{\bar{d}} \in D} \sum_{h \in H} y_{s\bar{\bar{d}}\bar{d}h} \qquad \forall s \in S, \forall \bar{d} \in D \tag{7}$$

$$\sum_{t \in T} v_{sdth} = \sum_{\bar{d} \in D} y_{s\bar{d}dh} \qquad \forall s \in S, \forall d \in D, \forall h \in H \tag{8}$$

$$\sum_{t \in T} \sum_{h \in H} t \cdot v_{sdth} + BM \left(1 - \sum_{t \in T} \sum_{h \in H} v_{sdth}\right) \geqslant Sk_{\bar{d}d} \left(\sum_{t \in T} \sum_{h \in H} t \cdot v_{s\bar{d}th}\right) \qquad \forall s \in S, \forall d \in D, \forall \bar{d} \in D \tag{9}$$

$$\sum_{d \in D} \sum_{\bar{d} \in D} y_{s\bar{d}dh} \leqslant \Lambda \qquad \forall s \in S, \forall h \in H \tag{10}$$

Requirements related to the hospitals

Constraints (11) and (12) formalise the minimum and maximum student staffing requirements respectively for each ward entity, for which a unique number of positions are postulated by the involved hospital. Constraint (13) ensures that the assignment of a student to a discipline and hospital is conform to the student ability and the rules set by the hospital for accepting students.

$$\sum_{s \in S} \sum_{d \in D} \sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot v_{sd\bar{t}h} \geqslant Dem^{Min}_{wht} \qquad \forall t \in T, \forall w \in W, \forall h \in H \tag{11}$$

$$\sum_{s \in S} \sum_{d \in D} \sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot v_{sd\bar{t}h} \leqslant Dem^{Max}_{wht} \qquad \forall t \in T, w \in W, h \in H \tag{12}$$

$$\sum_{t \in T} v_{sdth} \leqslant Ab_{sdh} \qquad \forall s \in S, \forall d \in D \setminus \{0\}, \forall h \in H \tag{13}$$

Requirements related to the medical students

Constraint (14) makes sure that the student is available during the time period(s) he/she is assigned to a particular discipline. Constraint (15) indicates if a student makes a change between hospitals, i.e. when the student-hospital assignment to perform discipline d differs from the student-hospital assignment related to the preceding discipline $\bar{d}$. Constraints (16) and (17) determine the waiting times between consecutive disciplines, which is calculated as the difference between the start time of discipline d and the start time plus the duration of the directly preceding discipline $\bar{d}$. Note that these waiting times are only correctly calculated if discipline d is directly preceded by discipline $\bar{d}$, such that these constraints rely on the network flow variables $y_{s\bar{d}dh}$ to make the constraint relevant or redundant. Constraint (18) calculates the attained desire score for each student. The desire score is composed of five components, i.e. (i) the student preferences related to a particular discipline, represented by Pr^{disc}_{sd}; (ii) the student preferences related to the hospitals in which they intend to carry out their training, indicated by Pr^{hosp}_{sh}; (iii) a more general preference for all students to carry out a particular discipline, reflected by Pr^{trPr}_{d}. Note that these preferences are strictly positive and are defined such that a higher preference leads to a higher student desire. Furthermore, we consider two undesirable components in this first objective, i.e. (iv) we minimise the total number of changes for students between different hospitals. Students changing between hospitals may be inevitable because of variable staffing requirements between disciplines and time periods or some hospitals may not provide a particular discipline; (v) we minimise the waiting time expressed in time periods between consecutive assignments. In order to compromise between these preference objectives, students can assign different weights to the different components such that individual students can realise a balanced trade-off. In this way, the aggregated preference score (Des_s) is calculated for every student building his/her desire towards the schedule. Note that in order to calculate the overall students' desire scores correctly, negative signs are allocated to the objectives minimising the total number of changes and the waiting time to aggregate these undesirable measures with the students preferences, which are maximised. Constraint (19) finds the student with the worst desire outcome across students. Constraints (20) state the domain of the variables.

$$\sum_{h\in H} Du_d \cdot v_{sdth} \leqslant \sum_{\bar{t}=t}^{t+Du_d-1} Av_{s\bar{t}} \qquad \forall s\in S, t\in T, d\in D \quad (14)$$

$$Ch_{sd} \geqslant y_{i\bar{d}dh} - \sum_{\bar{\bar{d}}\in D} y_{s\bar{\bar{d}}dh} \qquad \forall\, s\in S, \forall d,\bar{d}\in D, \forall h\in H \quad (15)$$

$$\sum_{t\in T}\sum_{h\in H} t\cdot v_{sdth} \geqslant W_{sd} + \sum_{t\in T}\sum_{h\in H} t\cdot v_{s\bar{d}th} + Du_{\bar{d}} - BM\left(1-\sum_{h\in H} y_{s\bar{d}dh}\right) \quad \forall s\in S, \forall d\in D, \forall\bar{d}\in D \quad (16)$$

$$\sum_{t\in T}\sum_{h\in H} t\cdot v_{sdth} \leqslant W_{sd} + \sum_{t\in T}\sum_{h\in H} t\cdot v_{s\bar{d}th} + Du_{\bar{d}} + BM\left(1-\sum_{h\in H} y_{s\bar{d}dh}\right) \quad \forall s\in S, \forall d\in D, \forall\bar{d}\in D \quad (17)$$

$$\begin{aligned} Des_s \leqslant & \sum_{\bar{d}\in D}\sum_{d\in D}\sum_{h\in H}\left(\omega_s^{disc}\cdot Pr_{sd}^{disc}\cdot y_{\bar{d}ddh}\right) + \sum_{\bar{d}\in D}\sum_{d\in D}\sum_{h\in H}\left(\omega_s^{hosp}\cdot Pr_{sh}^{hosp}\cdot y_{\bar{d}ddh}\right) \\ & - \sum_{d\in D}\left(\omega_s^{wait}\cdot W_{sd}\right) + \sum_{\bar{d}\in D}\sum_{d\in D}\sum_{h\in H}\left(\omega^{trPr}\cdot Pr_d^{trPr}\cdot y_{\bar{d}ddh}\right) \\ & - \sum_{d\in D}\left(\omega_s^{chng}\cdot Ch_{sd}\right) \qquad \forall s \quad (18) \end{aligned}$$

$$Des^{min} \leqslant Des_s \qquad \forall\, s \quad (19)$$

$$\begin{aligned} & y_{sddh} \in \{0,\ 1\} && \forall s\in S, \forall\bar{d}, d\in D, \forall h\in H \\ & v_{sdth} \in \{0,\ 1\} && \forall s\in S, \forall d\in D, \forall t\in T, \forall h\in H \\ & Ch_{sd} \in \{0,\ 1\} && \forall s\in S, \forall d\in D \\ & W_{sd} \geqslant 0 && \forall s\in S, \forall d\in D \\ & Des_s \geqslant 0 && \forall s\in S \\ & Des^{min} \geqslant 0 \end{aligned} \quad (20)$$

4. Solution methodology

In this study, we present a dedicated branch-and-price solution methodology that exploits different problem-specific mechanisms to reduce the size of the branching tree and to speed up the computational performance, enabling the procedure to solve larger-sized problem instances. The procedure relies upon the Dantzig-Wolfe decomposition of the compact problem formulation to tighten the linear programming relaxation. The decomposition is related to the scheduling of individual students, which is modelled by student pattern variables representing the assignment of students to individual schedules spanning the entire scheduling horizon. In this way, the original student scheduling variables and the feasibility of the associated student schedules are considered implicitly in the corresponding master problem. Due to the large number of student pattern variables, the solution procedure builds upon column generation to find the linear programming (LP) relaxation of the master problem (Section 4.1). The embedded pricing procedure applies different dedicated heuristics, pruning rules and a symmetry breaking rule to speed up the performance of the column generation procedure.

When the optimal LP solution in the root node is fractional, additional steps are required to drive this fractional solution to integrality. A branching tree is set up exploiting different types of dedicated branching rules based on the assigned capacity and the original decision variables, impacting both the restricted master problem and the student pricing problem to reach integer solutions (Section 4.2). The branching rules are applied in a specific structure in order to ensure a complete branching scheme. The unexplored nodes in the branching tree, stored in a node list, are visited in the order defined by a *right-first* search strategy, which exploits first the nodes with the largest number of student assignments fixed via the branching constraints (Section 4.3). At each node of the branching tree a column generation step is invoked to find the optimal LP solution, taking the branching constraints into account to derive adequate formulations for the restricted master problem and pricing problem. In every node, we calculate integer lower bound solutions for the maximisation problem under study in an attempt to improve the incumbent solution using two heuristics relying on the column variables with a strictly positive value in the optimal LP solution (Section 4.4). If the calculated lower bound in any node is not equal to the upper bound, i.e. the optimal LP solution, the search space of the restricted master problem is divided into two subproblems by a branching rule. Using this branch-and-price framework that is augmented by lower bound calculations, integer solutions are obtained and the incumbent solution can be updated. Note that we store all columns, generated via the pricing procedure, in a column pool that is shared over all nodes in the tree. In each node, the column generation initially starts with only those columns that satisfy the relevant branching constraints. The procedure terminates when the best known lower bound solution equals the best known upper bound solution, i.e. when the branching tree is fully explored, or when the prescribed time limit is reached. In the Online Appendix A, the different steps in the algorithm are presented by means of pseudocode.

4.1. Column generation

To solve the LP relaxation of the master problem using column generation, only a subset of all feasible student schedules are considered, defining the so-called restricted master problem (Section 4.1.1). A pricing problem is solved to check the optimality of an LP solution (Section 4.1.2), which tries to identify new student schedules with positive reduced cost to enter the basis taking dual information from the master problem into account. The pricing problem is solved using a two-step pricing procedure relying on a dedicated dynamic programming algorithm that aims to find a (diverse set of) schedule(s) with positive reduced cost (Section 4.1.3). The columns with a positive reduced cost are added to the column pool and the restricted master problem is re-optimised. When no such column exists, an optimal solution of the LP relaxation is found, which is an upper bound for the best integer solution contained in the node under study.

4.1.1. The restricted master problem

In the master problem, the original decision variables v_{sdth} and $y_{s\bar{d}dh}$ and the student scheduling constraints imposed on an individual student schedule are taken implicitly into account via the definition of so-called student pattern variables. A student pattern variable represents a student schedule, listing the disciplines allocated to a student over the scheduling horizon. Hence, in the master problem each feasible student schedule is represented by a pattern $j\in J$, where the set J represents the set of all feasible patterns. The variable X_{sj} takes the value 1 if student schedule j is selected for student s and 0 otherwise. In a column generation procedure, the restricted master problem is defined, which considers only a subset of those variables by introducing the set $\bar{J}\subset J$ and is formulated as follows.

Additional notation

Sets

J	The set of student schedules (index j)
$\bar{J}$	The set of student schedules considered in the restricted master problem (index j)

Parameters

$\overline{Des}_{sj}$	The aggregated desire score for student s attained in schedule j
$\bar{v}_{sdthj}$	1, if schedule j for student s sets the start time for discipline d in hospital h to time period t; 0, otherwise
$\bar{y}_{s\bar{d}dhj}$	1, if schedule j for student s indicates that discipline d in hospital h follows directly after discipline $\bar{d}$; 0, otherwise

Decision variables

X_{sj}	1, if student s is assigned to schedule j; 0, otherwise

Mathematical formulation

$$\text{Maximise } Z = \alpha \sum_{s\in S}\sum_{j\in \bar{J}} \overline{Des}_{sj} \cdot X_{sj} + \beta Des^{min} \tag{21}$$

$$\text{s.t.} \sum_{j\in \bar{J}} X_{sj} = 1 \qquad \forall s \in S \tag{22}$$

$$\sum_{s\in S}\sum_{d\in D}\sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot \bar{v}_{sd\bar{t}hj} \cdot X_{sj} \geqslant Dem^{Min}_{wht} \qquad \forall w \in W, \forall h \in H, \forall t \in T \tag{23}$$

$$\sum_{s\in S}\sum_{d\in D}\sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot \bar{v}_{sd\bar{t}hj} \cdot X_{sj} \leqslant Dem^{Max}_{wht} \qquad \forall w \in W, \forall h \in H, \forall t \in T \tag{24}$$

$$Des^{min} \leqslant \sum_{j\in \bar{J}} \overline{Des}_{sj} \cdot X_{sj} \qquad \forall\, s \in S \tag{25}$$

$$X_{sj} \in \{0,\ 1\} \qquad \forall s \in S, \forall j \in \bar{J}$$
$$Des^{min} \geqslant 0 \tag{26}$$

The objective function (21) of the restricted master problem is a maximisation function that embodies a weighted function of the sum of the student desires and the worst desire score across students and is equivalent to equation (1). Constraints (23) and (24) formalise the minimum and maximum student staffing requirements for each ward. Constraint (25) embodies the fairness criterion and determines the quality associated with the worst schedule across students. Constraints (26) embody the domain constraints for the variables included in the restricted master problem. Note that in a column generation procedure, these domain constraints are relaxed, i.e. $0 \leqslant X_{sj} \leqslant 1$ ($\forall s \in S, \forall j \in \bar{J}$), to find the optimal LP solution.

4.1.2. The student pricing problem

In order to guarantee that the optimal solution of the restricted master problem corresponds to an optimal LP solution of the problem under study, we try to identify feasible student schedules with positive reduced cost by solving the pricing problem for each student. The formulation of the pricing problem for a particular student s relies on the original decision variables v_{sdth} and $y_{s\bar{d}dh}$ and considers the educational requirements set by the medical school (eqs. (2)–(10)), the ability requirement set by the hospitals (eq. (13)), the requirements related to the students (eqs. (14)–(18)) and the domain constraints (eqs. (20)). The reduced cost (RC_s) is based on the student desire and the dual variables stemming from the constraints in the restricted master problem. Let $\pi^{22}_s, \pi^{23}_{wht}, \pi^{24}_{wht}$ and π^{25}_s be the respective dual prices associated with constraints (22), (23), (24) and (25). The objective function of the pricing problem (27) for student s maximises the reduced cost RC_s and is formulated as follows

$$\text{Maximise } RC_s = Des_s - \pi^{22}_s - \sum_{w\in W}\sum_{h\in H}\sum_{t\in T}\pi^{23}_{wht}\sum_{d\in D}\sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot v_{sd\bar{t}h} - \sum_{w\in W}\sum_{h\in H}\sum_{t\in T}\pi^{24}_{wht}\sum_{d\in D}\sum_{\bar{t}=t-Du_d+1}^{t} \xi_{dwh} \cdot v_{sd\bar{t}h} + \pi^{25}_s \cdot Des_s \tag{27}$$

4.1.3. The pricing problem solution method

The procedure to solve the pricing step iterates over all individual students. The underlying structure of a singular pricing problem is a resource-constrained shortest path problem, which is solved using *dynamic programming* (DP) to build feasible schedules while maximising the reduced cost. In order to speed up the computational performance of the DP algorithm, we employ different *pruning rules* and a *symmetry breaking rule* to limit the number of nodes visited. The pricing solution method produces columns with a positive reduced cost via a two-step procedure. In a first step, we generate a diverse set of suitable columns quickly based on a *heuristic DP algorithm*. In this step, we store for every student a predefined number of diverse columns with positive reduced cost in a solution pool during the course of the DP algorithm. All the stored columns in this solution pool are added to the restricted master problem. In a second step, the pricing problem is solved exactly, able to return the column with the most positive reduced cost. However, in our procedure we stop the algorithm from the moment the first column with a positive reduced cost has been found.

4.1.3.1. Dynamic programming: General principles. Similar to the DP algorithm proposed by Akbarzadeh and Maenhout (2020), we rely on the general logic of assigning a *student* to a discipline and hospital (i.e. an available *position*) at a particular *time period*. A stage corresponds to a single period. We define $PList_s$ as the ordered list of positions that has been assigned to student s in previous stages. Suppose we are assigning at a particular stage student s to the position pos (i.e. discipline d at hospital h) after position $\overline{pos}$, which has been assigned in the previous stage. We use the notation $pos.d$ to represent the discipline related to position pos and $pos.h$ to denote the involved hospital. Based on the list $PList_s$, the start time for position pos is equal to $CT(PList_s) = \left(\sum_{d\in PList_s} Du_d + \sum_{d^w\in PList_s} Du_{d^w}\right)$. Note that a visited waiting position, denoted as d^w, is also added to $PList_s$ with $Du_{d^w} = 1$.

The problem at a particular stage can be described as finding the position to be added to the list that has the highest reduced cost. Given the ordered list of positions $PList_s$ for student s and the last visited position $\overline{pos}$ in this list, the associated reduced cost $RCP(PList_s, \overline{pos}, pos)$ for assigning student s to the new position pos is calculated via Eq. (28).

$$RCP(PList_s, \overline{pos}, pos) = \left(\alpha + \pi^{25}_s\right) \cdot Des_s(\overline{pos}, pos) - RCS(pos.d,\ pos.h,\ CT(PList_s)) \tag{28}$$

The different terms within Eq. (28) are explained as follows:

- The desire $Des_s(\overline{pos}, pos)$ to assign student s to position pos after position $\overline{pos}$ is calculated via Eq. (29).

$$Des_s(\overline{pos}, pos) = \omega_s^{disc} \cdot Pr_{s,pos.d}^{disc} + \omega_s^{hosp} \cdot Pr_{s,pos.h}^{hosp} + \omega^{trPr} \cdot Pr_{pos.d}^{trPr} + \omega_s^{chng} \quad \text{if } (\overline{pos}.h \neq pos.h) + \omega_s^{wait} \quad \text{if } (pos.d = d^w) \tag{29}$$

- $RCS(d,h,t)$ calculates the reduced cost stemming from the dual prices of the hospital demand constraints (Eqs. (23) and (24)) if student s is assigned to discipline d in hospital h at time t. $RCS(d,h,t)$ is calculated by Eq. (30).

$$RCS(d,\ h,\ t) = \sum_{w \in W} \sum_{\bar{t}=t}^{t+Du_d-1} \xi_{dwh} \cdot (\pi_{\bar{t}wh}^{23} + \pi_{\bar{t}wh}^{24}) \tag{30}$$

Given the cost $RCP(PList_s, \overline{pos}, pos)$ associated with arc $(\overline{pos}, pos)$, we can formulate a recursive expression for $RC(PList_s, pos)$ as follows

$$RC(pos, PList_s) = \min_{\overline{pos}}\{RC(\overline{pos}, PList_s \setminus \{\overline{pos}, pos\}) + RCP(PList_s, \overline{pos}, pos)\} \tag{31}$$

According to eq. (31), the cost for adding the new position pos to the schedule for student s is only dependent on the previous position $\overline{pos}$ and the start time of the new position (i.e. $CT(PList_s)$). Note that $CT(PList_s)$ is independent from the ordering of the positions in $PList_s$. When a student schedule is completed, we subtract the term π_s^{22} from $RC(pos, PList_s)$ to determine the reduced cost of the column.

4.1.3.2. Pruning rules. We include the following pruning rules (PR) to limit the number of nodes, i.e.

PR1. *An optimal decision in any state for a student is independent of the sequence of previously assigned student positions or discipline-ward-hospital assignments. Considering the sequence with the lowest cost is sufficient* (Akbarzadeh and Maenhout, 2020).

PR2. *Since waiting times have been potentially scheduled between consecutive discipline assignments, it may occur that a student has insufficient time left in order to complete his/her training program. As a result, a feasibility check related to the duration can be performed in any state by calculating the minimum required time to complete the schedule. The latter is based on the minimum duration of the required number of disciplines from each discipline group. Whenever the minimum required time to complete the schedule exceeds the remaining horizon, the node is pruned.*

PR3. *In any state an upper bound related to the maximum attainable reduced cost can be calculated for any incomplete path. The bound is computed by adding the disciplines with the largest reduced cost to the incomplete path until it has been completed. When this bound is not positive, the node can be pruned.*

4.1.3.3. Symmetry breaking rule. Discipline d and discipline $\overline{d}$ for a medical student s are called symmetric disciplines if (i) they have same duration, (ii) they are included in the same group or groups of disciplines, (iii) the student has a higher or equal preference for discipline d than for discipline $\overline{d}$ and, (iv) $\overline{d}$ is not a predecessor for discipline d. Based upon this notion of symmetric disciplines, we introduce a symmetry breaking rule (*SymB*) to reduce the size of the DP tree:

SymB. *When disciplines d and $\overline{d}$ are symmetric for student s and $RCS(d,h,t) = RCS\left(\overline{d},h,\overline{t}\right)$ $(\forall\ h \in H, \forall t, \overline{t} \in T)$, discipline d is always scheduled before discipline $\overline{d}$ in a given hospital h, i.e. $\sum_{t \in T} t \cdot v_{stdh} < \sum_{t \in T} t \cdot v_{st\overline{d}h}$.*

Suppose that discipline d and discipline $\overline{d}$ are symmetric and they are the only two disciplines that a student must follow to complete his/her training program. In that case, there are two potential sequences for these two disciplines, either the student starts with discipline d or with discipline $\overline{d}$. The impact of an interchange between these two disciplines in the sequence appears in the calculated measure $RCS(d,h,t)$ whereas the rest of the reduced cost is independent of the sequence between these two disciplines. When $RCS(d,h,t) = RCS\left(\overline{d},h,\overline{t}\right)$, the reduced cost is equal for both sequences. Therefore without loss of optimality we prune the nodes that schedule discipline $\overline{d}$ before discipline d.

Proposition. *For the symmetric disciplines d and $\overline{d}$ and the subset of time periods $\overline{T}$, if $RCS(d,h,t) = RCS\left(\overline{d},h,\overline{t}\right)(\forall\ h \in H,\ \forall t, \overline{t} \in \overline{T})$ and discipline d starts at $St_d = \sum_{t \in \overline{T}} t \cdot v_{stdh}$, discipline $\overline{d}$ can not start at $St_{\overline{d}} = \sum_{t \in \overline{T}} t \cdot v_{st\overline{d}h} | St_d > St_{\overline{d}}$.*

4.1.3.4. Heuristic pricing. The first step of the pricing procedure applies the proposed DP procedure in a heuristic manner in order to try to efficiently find a diverse pool of suitable columns with a positive reduced cost following the methods described below.

The operating principles - The heuristic DP works based on a postulated time limit and the hospital demand (i.e. available positions) to direct the student schedules to preferable columns. This heuristic approach solves the pricing problem student by student and updates the remaining capacity of the involved disciplines and hospitals after a student has been scheduled. In this way, this heuristic will create columns for all students in line with available capacity prohibiting infeasible solutions, taking information from the master problem into account. Due to the limited number of student positions, the order in which the heuristic visits the students has a significant impact. We sort the students based on the ascending order of $\sum_{j \in J} X_{sj}$ $(\forall s \in S)$ in each iteration of the column generation step. Note that in the first few iterations not all students may be assigned to exactly one schedule $(\exists\ s \in S | \sum_{j \in J} X_{sj} < 1)$ due to the hard constraints related to the hospital capacity. As a result, Eq. (22) is relaxed and a deviation variable is included that is largely penalised in the objective function. The heuristic dynamic programming runs only for a predefined time and is terminated when this time limit is reached.

Construction of a diverse pool of solutions - In the course of the DP procedure, we may find a feasible schedule with positive reduced cost. Such a column will be saved in a solution pool of limited size, which is added to the restricted master problem in the next column generation iteration. The size of this solution pool depends on the number of possible permutations of disciplines, hospitals and time periods $(|D| \cdot |H| \cdot |T|)$ and the parameter ps. This solution pool size decreases during the column generation process because in the early iterations of the column generation step it is important to find many feasible, suitable columns with positive reduced cost, which is much easier than in later iterations.

In order to ensure that sufficiently dissimilar columns for a particular student are stored in the solution pool, we prune the nodes that will lead to very similar columns based on a threshold value. This threshold value is dependent on the number of solutions that are at some point already stored in the solution pool (#*sol*) and is equal to $\left(1 - \frac{1}{\#sol+1}\right) \cdot 100\%$. In this way, the second column should be at least 50% different from the first found column, the third column should be 67% different, etc. The resemblance between two columns is measured via the Hamming distance, which is reflected in the structure of the DP tree. Paths yielded by the DP algorithm that show a higher resemblance than the threshold value are pruned.

4.2. Branching Strategy

We apply a mixed-shifting branching strategy to set up a branching tree to derive the optimal integer solution, which

combines different binary branching rules (Section 4.2.1) applied in a specific branching order (Section 4.2.6).

4.2.1. Branching rules

We explore different branching rules to drive the decision variables to integrality leading to branching constraints that are added to the restricted master problem and/or the pricing problem. The branching is based on the optimal LP solution resulting from the restricted master problem. The proposed branching rules are the following, i.e.

Branching on allocated ward capacity per time period (BR1)

We calculate the number of students allocated to a particular ward, hospital and time period, i.e. $\sum_{s\in S}\sum_{d\in D}\sum_{\bar{t}=t-Du_d+1}^{t}\xi_{dwh}\cdot\bar{v}_{s\bar{d}\bar{t}hj}\cdot X_{sj}=xDem_{wht}$ $(\forall w\in W,\forall h\in H,\forall t\in T)$. The tuple (w,h,t) with a fractional value for $xDem_{wht}$ that is selected for branching, is determined as follows. The ward w is selected based on the largest minimum duration of the associated disciplines (i.e. $\min_d\{\xi_{dwh}\cdot Du_d\}$). The higher the duration of the related discipline, the more effective the branching. The hospital h and time period t are selected based on the largest fractional value for the calculated measure $xDem_{wht}$. In order to tighten the convex hull of the (restricted) master problem, we add standard integer branching constraints to the master problem, creating two disjoint subproblems that restrict the number of students allocated to ward w, hospital h and time period t to be either (i) smaller than or equal to $\lfloor xDem_{wht}\rfloor$ (left branch) or (ii) larger than or equal to $\lceil xDem_{wht}\rceil$ (right branch). This branching scheme will not lead per definition to an integer student roster as it may occur that the values of the student schedules are fractional but that the resulting workload is integer for all tuples (w,h,t) and this branching rule cannot be executed.

Branching on allocated capacity per hospital (BR2)

We calculate the number of students allocated to a particular hospital summed over all wards and time periods, i.e. $\sum_{w\in W}\sum_{t\in T}xDem_{wht}$ $(\forall h\in H)$ and select the hospital h with the largest fractional value. When this total number of students allocated to hospital h is fractional, we add standard integer branching constraints to the master problem to create two subproblems that restrict the number of students allocated to hospital h to be either (i) smaller than or equal to $\lfloor\sum_{w\in W}\sum_{t\in T}xDem_{wht}\rfloor$ (left branch) or (ii) larger than or equal to $\lceil\sum_{w\in W}\sum_{t\in T}xDem_{wht}\rceil$ (right branch). Similarly to branching rule BR1, this branching rule delivers not a complete branching scheme.

Branching on network flow decision variable (BR3)

We consider the original network flow decision variable $y_{s\bar{d}dh}$, relevant to the pricing problem, i.e. $y_{s\bar{d}dh}=\sum_{j\in\bar{J}}\bar{y}_{s\bar{d}dhj}\cdot X_{sj}$ $\left(\forall s\in S,\forall\bar{d},d\in D,\forall h\in H\right)$. Assuming that the value of $y_{s\bar{d}dh}$ is fractional, we add branching constraints to the pricing problem, creating two disjoint subproblems that either exclude (left branch) or include (right branch) the direct precedence between two disciplines. This branching rule selects the branching variable $y_{s\bar{d}dh}$ according to the following principles. We determine the relevant student by identifying the student schedule that has the largest fractional value X_{sj} $(\forall s\in S,\forall j\in\bar{J})$. For the selected student, we calculate the (fractional) value that discipline $\bar{d}$ is followed by discipline d, i.e. $\sum_{j\in\bar{J}}\sum_{h\in H}\bar{y}_{s\bar{d}dhj}\cdot X_{sj}$ $\left(\forall\bar{d},d\in D\right)$. We select the discipline d with the largest number of strictly positive values and choose the pair of disciplines $(\bar{d},d)$ with the largest fractional value. Only when the student schedule with the largest fractional value X_{sj} includes the direct precedence between disciplines $\bar{d}$ and d (i.e. $\bar{y}_{s\bar{d}dhj}=1$ $(\exists h\in H)$), we select the related variable $y_{s\bar{d}dh}$ as branching variable. Otherwise, we continue by visiting discipline d with the next largest number of strictly positive values. Note that this branching rule does not provide a complete branching scheme as a result of potential waiting times between disciplines.

Branching on the original assignment variable (BR4)

We consider the original start time decision variables v_{stdh}, relevant to the pricing problem, i.e. $v_{stdh}=\sum_{j\in\bar{J}}\bar{v}_{stdhj}\cdot X_{sj}$ $(\forall s\in S,\forall t\in T,\forall d\in D,\forall h\in H)$. We select the variable v_{stdh} that has the largest fractional value and add branching constraints to the pricing problem to create two disjoint subproblems either excluding (left branch) or including (right branch) the specific student-discipline assignment. Note that this branching rule provides a complete branching scheme as an integer value for all original assignment variables comprises an integer solution for the master problem.

4.2.2. Branching order between the branching rules

In order to devise a proper branching strategy, an ordering is required to denote when to apply which branching rule. This order can be based on a random selection or can be directed by some problem-specific knowledge to create a more efficient branching order. To that purpose, we propose to first branch on the capacity branching rules BR1 followed by BR2. If we fail to create a capacity branching constraint, we branch on the original decision variables, i.e. first on the precedence-related network flow variables (BR3) followed by the time-based assignment variables (BR4). However, preliminary computational results indicated that it is not efficient to keep on branching using branching rules BR1 and BR2 since the underlying branching schemes are not complete. As a result, we never apply the same branching rule twice in a consecutive manner unless we are forced to do so using the complete branching rule BR4. In addition, we alternate between capacity branching constraints (BR1 and BR2) and the branching constraints on the original decision variables (BR3 and BR4). Hence, if at some point we use branching rule BR1, the next branching rule we apply is either BR3 or BR4 followed by BR2.

4.3. Search strategy

The order in which the unexplored nodes in the branching tree are visited depends upon the search strategy. We employ a novel so-called *right-first* search strategy. All branching rules apply a branching scheme that divides the search space in two disjoint subproblems. Each time when a branching rule is applied, the one subproblem fixes the branching variable to a value smaller than its fractional value in the optimal LP solution rounded down to the closest integer (left branch) whereas the other subproblem fixes the branching variable to a value larger than its fractional value rounded up to the closest integer (right branch). This right-first strategy selects the node with the largest number of right branches, which directs the solution towards a student roster set according to the upper values imposed by the different branching constraints such that a larger number of assignments or arcs between successive disciplines are fixed. In case of a tie, we employ a best-first search strategy, which is common in literature and selects the node with the largest upper bound value resulting from the calculated LP relaxations.

4.4. Lower bounds

In order to improve the computational performance, we try to find a (high-quality) integer solution in an efficient manner in

every node of the branching tree. These lower bound solutions are derived based on the optimal LP solution resulting from the column generation algorithm and are developed using two heuristics, i.e. a greedy heuristic (Section 4.4.1) and a Hungarian-based heuristic (Section 4.4.2). Both heuristics build an integer solution in two steps. In a first step, the information of the optimal LP solution in the considered node is collected and refined. In a second step, this information is used to construct an integer feasible roster.

4.4.1. Greedy lower bound

The *Greedy Heuristic* uses the information of the optimal LP solution by finding the column/student schedule with the highest value for the related decision variable X_{sj}. When the (remaining) hospital capacity allows the assignment of the student positions as indicated by the selected column, the student schedule will be assigned to the involved student. If there are not enough resources to schedule the student to the indicated positions, the selected column will be removed. In a following iteration, the next column with the highest value for the related decision variable will be selected. Whenever a student schedule has been assigned to a student, all columns related to that student will be removed and are not further considered in this constructive heuristic. Note that, only those columns with a strictly positive value ($X_{sj} > 0$) are considered. This heuristic stops whenever there are no columns left and does not ensure the construction of a feasible integer solution.

4.4.2. Hungarian-based lower bound

The *Hungarian-based Heuristic* constructs a roster period by assigning all the students to feasible ward-hospital positions over the different time periods in a chronological order (Akbarzadeh and Maenhout, 2020). The underlying structure of a subproblem, which is related to a single period, embodies an assignment problem (Ahuja et al., 1993) that assigns students to available positions during a particular period minimising the total assignment cost and is solved using the Hungarian algorithm (Kuhn, 1955). The assignment cost is influenced by (i) the student desire for a particular position and (ii) a directional cost associated with the value of the original decision variable v_{stdh}, derived from the optimal LP solution of the considered node via the restricted master problem. When the cost matrix considers both these elements, the constructive heuristic deploys not only the information of the optimal LP solution, but also uses its own intelligence to build a high-quality integer lower bound solution by assigning the students to positions based on the desire objective.

5. Computational experiments

In this section, we provide computational insights into the proposed procedure. In Section 5.1, we describe the relevant parameter settings and the methodology based on which a synthetic dataset of test instances is constructed. In the next sections, we validate the computational impact of the different aspects of the proposed methodology. In Section 5.2, we analyse the applied branching rule and search strategy. Section 5.3 discusses the performance of the lower bound calculations. In Section 5.4, we investigate the performance of the proposed two-step pricing approach and benchmark with other pricing methodologies. Section 5.5 benchmarks the proposed procedure with a compact MIP model solved via the commercial solver CPLEX and a standard branch-and-price procedure. In addition, we show the contribution of the different algorithm components in a gradual manner. The proposed algorithm is coded in C# using IBM ILOG/OPL 12.9, and all tests are carried out on a Windows PC with Corei7 - 3.40 GHz CPU and 8 GB RAM.

5.1. Test design

In order to investigate the (stable) performance of the proposed solution approach under different settings, we compose a set of problem instances in a systematically varied and controlled way in order to devise general findings. Note that most of the parameters are determined based upon preliminary experiments and the observation and analysis of real-life practice and academic literature. The dataset is generated to represent problem instances with the largest complexity for the problem under study, defining a substantial number of students, degree of decision freedom to elect disciplines and number of precedence relationships between disciplines (Ito et al., 2017; Akbarzadeh and Maenhout, 2020). As a result, different groups of instances are created based on different levels for the parameters indicated in Table 1. These parameters have an impact on the problem size and problem complexity. Based upon these characteristics, we determined $2 \cdot 2 \cdot 3 = 12$ groups and devised 5 instances per group, leading to 60 instances. The dataset is publicly available on the Mendeley Data repository (doi:10.17632/x5pj3kn6rv.1).

University

We assume a training program with a one-year horizon, which contains 12 time periods ($|T| = 12$) of 4 weeks. The number of disciplines is equal to 12 ($|D| = 12$). All students are obliged to carry out a number of disciplines equal to $|D| \cdot R_{D,\kappa}$. The program is organised in a mandatory and an elective group of disciplines ($|G| = 2$). Each student must fulfil all the courses from the mandatory group, which embodies 40% of the training program, and the remainder of the disciplines, 60% of the training program, can be elected from the elective course list. The duration of each discipline is randomly chosen from the range of $\left[1, \lceil \frac{|T|}{R_{D,\kappa} \cdot |D|} \rceil\right]$. Hence, the minimum duration equals one period. The maximum duration is equal to the number of time periods divided by the number of disciplines the student needs to carry out.

In order to define indirect precedence relations between disciplines ($Sk_{\bar{d}d}$), we employ three parameters, i.e. the type of relationship (*PrRl*), the maximum number of progressive levels in a precedence relationship between disciplines (equal to $R_{\#PL,D} \cdot |D|$) and the total number of precedence relationships between disciplines in the training program (equal to $R_{\#PrRl,D} \cdot |D|$). In order to generate a precedence relationship for relationship $PrRl = 1.1$, we select a discipline randomly (e.g. d_3). We connect the selected discipline to one other randomly chosen discipline (e.g. d_5) and define a precedence relationship between both these disciplines (e.g. $d_3 \rightarrow d_5$). Additional precedence relationships are added to this

Table 1
Controlled parameter settings in the test design.

Parameter	Description	Levels
$\|S\|$	The total number of students	40, 80
$R_{D,\kappa}$	The percentage of disciplines that a student must follow in the training program	0.60, 0.75
PrRl	The parameter defining the type of precedence relationship, i.e. (1.1): one predecessor is defined for a specific discipline (2.1): two different predecessors are defined for a specific discipline (1.2): one predecessor is defined for two different disciplines	1.1, 1.2, 2.1
$R_{\#PL,D}$	The parameter defining the maximum number of progressive levels in a precedence relationship between disciplines	0.2
$R_{\#PrRl,D}$	The parameter defining the maximum number of precedence relationships between disciplines	0.3

sequence of disciplines (e.g. $d_3 \rightarrow d_5 \rightarrow d_6$) as long as the maximum number of progressive levels in the precedence relationship between disciplines or the maximum number of precedence relationships between disciplines is not exceeded. Note that the precedence relationships between disciplines are generated in a similar manner for the other relationships $PrRl = 1.2$ or 2.1 for which respectively two successors or two predecessors are randomly selected. No restrictions are imposed on the number of disciplines that can be carried out in a single hospital ($\Lambda = |D|$).

Hospitals

Each ward is associated with a single discipline, which implies that the number of wards $|W|$ is equal to $|D|$. In addition, all disciplines can be carried out in all hospitals ($\xi_{dwht} = 1$). The number of hospitals is set equal to $|H| = \frac{|S| \cdot R_{D,K} \cdot |D|}{|T| \cdot |D|}$ in which the nominator represents the total number of positions required by all students and the denominator shows the total available number of positions per hospital. The student staffing requirements are determined in a random manner. The maximum staffing requirement Dem^{Max}_{wht} is set randomly in the range [1, 5]. The minimum staffing requirement Dem^{Min}_{wht} is set to 1 with a 5% probability and is 0 otherwise.

Medical students

We assume that all students are available during every time period of the scheduling horizon (Av_{st} = 1) and are able to perform all disciplines in the different hospitals ($Ab_{sdh} = 1$).

Objective function

The student preferences $Pr^{disc}_{sd}, Pr^{hosp}_{sh}$ and Pr^{trPr}_{d} are selected randomly from the interval $[1, 4]$. The weights to calculate the weighted student desires ($\omega^{disc}_s, \omega^{hosp}_s, \omega^{trPr}, \omega^{chng}_s$ and ω^{wait}_s) are randomly generated in a range of $[1, 4]$. In order to aggregate the two main components of the objective function (Eq. (21)), an equal weight setting has been applied, i.e. the objective function coefficients α and β are set to 1. In this way, the desire score of the worst schedule is accounted twice in the objective function. Note that this equal weight setting has been found in Akbarzadeh and Maenhout (2020) to lead to a balanced trade-off and has been approved by the practitioners of the visited real-life case study.

Experimental conditions

The time limit of the algorithm is set at $|S| \cdot |D| \cdot |T|$ seconds per instance, i.e. when $|S|$ = 40 (80) the time limit equals $40 \cdot 12 \cdot 12 = 5760$s ($80 \cdot 12 \cdot 12 = 11520$s). No time limit is imposed to truncate the column generation algorithm, such that the procedure stops due to this time limit when the optimal LP solution of the last considered node has been calculated. In the following, we perform computational experiments to discern the beneficial impact of the proposed optimisation and speed-up mechanisms. To that purpose, each analysis is performed in a ceteris paribus manner, meaning that the parameters not included in the analysis are set at their best values, which will become clear throughout the following sections. The following symbols are used to describe the computational results averaged over the set of instances, i.e.

Table 2
Computational impact of the search strategy

	depth-first	*best-first*	*right-first*	*integral-first*
$\#Itr_{DP}$	14	12	12	11
$\#Br_{DP}$	11	11	7	10
$CPU_{DP,-}$	299	242	166	219
$Gap_{DP,-}$	0.0%	0.0%	0.0%	0.0%
Opt_{DP}	95.0%	98.3%	100.0%	100.0%

Gap_x	the final optimality gap obtained when relying on procedure x
$CPU_{x,y}$	the required computational time in seconds for procedure x, to solve component y
$Opt_{x,n}$	the percentage of instances solved to optimality within the time limit using procedure x, in node n
$\#Itr_x$	the total number of column generation iterations realised when relying on procedure x
$\#ClC_x$	the total number of columns created using procedure x
$\#Br_x$	the total number of branches devised in the branching tree for procedure x
with $x = DP$	the pricing solution methodology based on dynamic programming
$x = CP$	the pricing solution methodology based on constraint programming
$x = MP$	the pricing solution methodology based on mathematical programming (MIP)
with $y = MP$	the master problem
$y = PP$	the pricing problem
$y = -$	the entire branch-and-price procedure
with $n = RN$	the root node
$n = -$	the complete branching tree

Unless otherwise stated, the tables in the subsequent sections denote the aggregate results averaged over the generated 60 instances. In order to show the impact of the problem size on the component-wise analysis of the design choices, we have added the separate computational results, listed according to the number of students, to the Online Appendix B. Note that the results for specific problem sizes confirm the findings based on the results averaged over different settings for the number of students. In addition, on top of these experiments to validate the performance of the algorithm, we have explored the computational sensitivity related to the problem dimensions, i.e. the number of students ($|S|$), disciplines ($|D|$) and time periods ($|T|$). The results are included in the Online Appendix C and reveal that (i) the procedure is still able to find (near-) optimal solutions for large-scale instances (with $|S|$ = 80, $|D|$ = 24, $|T|$ = 48) with a reasonable optimality gap and acceptable time span; (ii) increasing the number of disciplines has the largest computational impact.

5.2. The search structure in the branching tree

5.2.1. Search strategy

In order to evaluate the impact of the proposed search strategy (see Section 4.3), we analyse different versions of the algorithm implementing different search methods, i.e. *depth-first* search, *best-first* search, *right-first* search and *integral-first* search. The latter selects the node with the largest number of integer assignments to explore first. The results are displayed in Table 2.

Table 2 reveals that the proposed *right-first* branching strategy, which selects the nodes with the largest number of right branches, yields the best performance while the *depth-first* strategy leads to the worst performance. The *best-first* search strategy, which is commonly applied in the branch-and-price literature, leads to a larger CPU time ($CPU_{DP,-}$) and cannot close the optimality gap for all instances (Opt_{DP} = 98.3%). Note that the *right-first* and the *integral-first* search strategies are to some extent very similar, since fixing the variable to the upper values set by the branching constraints usually leads to a larger number of integral variables. These strategies leads to the best computational

performance ($CPU_{DP,-}$) and are able to solve all instances to optimality (Opt_{DP} = 100%).

5.2.2. Branching rules and ordering

We analyse the proposed mixed-shifting branching rule (see Section 4.2) by composing different branching variants to evaluate the effectiveness of the individual branching rules by omitting one specific branching rule. In addition, we assess whether the proposed branching order alternating between capacity branching constraints and branching constraints on the original decision variables (ALT) outperforms a random branching order (RND). We devised the following strategies, i.e. excluding BR1 with alternating branching order ('*w/o BR1 - ALT*'), excluding BR2 with alternating branching order ('*w/o BR2 - ALT*'), excluding BR3 with alternating branching order ('*w/o BR3 - ALT*'), all branching rules with alternating branching order ('*w All - ALT*') and all branching rules applied in a random order ('*w All - RND*'). The results are displayed in Table 3. Note that in Section 5.5.2 comparison is made with the singular branching on the original assignment variable (BR4), which is the standard branching rule in branch-and-price frameworks.

Overall, Table 3 shows that the removal of a hospital capacity branching rule ('*w/o BR1 - ALT*' and '*w/o BR2 - ALT*') deteriorates the performance of the algorithm in terms of the solution quality ($Gap_{DP,-}$) and in particular the CPU time ($CPU_{DP,-}$). The largest impact is observed when the branching on the hospital workload ('*w/o BR2 - ALT*') is left out of the algorithm as the total number of branches in the tree increases, which leads to a higher CPU time ($CPU_{DP,-} = 293$s). In contrast, removing the precedence-based branching rule on the network flow variables ('*w/o BR3 - ALT*') improves the performance of the branch-and-price algorithm ($CPU_{DP,-} = 166$s) since the order in which the different disciplines are carried out is of minor importance. In all other computational experiments in this paper, we applied this best performing version ('*w/o BR3 - ALT*').

Last, we compare the proposed alternating branching order with a random branching order. The results revealed that a random order ('*w All - RND*') performs significantly worse compared to an alternating order both in terms of CPU time ($CPU_{DP,-}$ = 630s) and solution quality as the algorithm fails to close the gap for some instances in the given time limit. Note that a logical branching order that tries to apply the branching rules in the same, consistent order from BR1 to BR4 for every node of the branching tree leads to a detrimental performance, which is not reported in the table, characterised by a very large depth of the branching tree.

5.3. Lower bounds

In order to evaluate the contribution of lower bound calculations (see Section 4.4), we composed different versions of the proposed algorithm to evaluate the effectiveness of both lower bounds and whether these lower bounds should be calculated only in the root node (RN) or in all nodes (AN) of the branching tree. Table 4 displays the impact for different compositions of the proposed lower bound heuristics, i.e. without any lower bound ('*w/o All*'), the greedy heuristic applied in all nodes ('*Greedy - AN*'), the Hungarian-based heuristic applied in all nodes ('*Hungarian - AN*'), both lower bounds applied only in the root node ('*w All - RN*') and both lower bounds applied in all nodes ('*w All - AN*').

Table 3
Computational impact of different branching rules and branching order.

	w/o BR1 - ALT	*w/o BR2 - ALT*	*w/o BR3 - ALT*	*w All - ALT*	*w All - RND*
$\#Itr_{DP}$	13	15	12	12	15
$\#Br_{DP}$	9	14	7	7	9
$CPU_{DP,-}$	228	293	166	186	630
$Gap_{DP,-}$	0.0%	0.1%	0.0%	0.0%	0.2%
$Opt_{DP,-}$	100.0%	98.3%	100.0%	100.0%	98.3%

Table 4 reveals that the upper bound calculations performed in every node of the branching tree are justified from a computational point-of-view. Omitting these lower bound calculations ('*w/o All*') leads to a significantly higher average CPU time of 788s as a result of the larger number of created branches ($\#Br_{DP}$) and the associated number of column generation iterations ($\#Itr_{DP}$). In addition, the proposed procedure is not able to find the optimal solution for all instances within the time limit ($Opt_{DP,-}$ = 93.3%).

The Hungarian-based lower bound ('*Hungarian - AN*') yields a better performance compared to the greedy heuristic ('*Greedy - AN*') not only in terms of CPU time but also in terms of optimality gap. For 33% of the instances the Hungarian-based lower bound is able to derive the optimal solution already in the root node of the branching tree ($Opt_{DP,RN}$), whereas the greedy heuristic achieves this only for 3% of the instances. When both the heuristics are included ('*w All - RN*' and '*w All - AN*'), 35% of the instances are solved to optimality in the root node. Omitting both lower bounds ('*w/o All*') reduces this value to only 2%. However, when these bound calculations are only applied in the root node ('*w All - RN*'), the computational effort required to solve the instances to optimality amounts 701s. Applying both the greedy and Hungarian-based heuristic in every node of the branching tree ('*w All - AN*') ensures that all instances are solved to optimality within the time limit and reduces the CPU time dramatically to 166s. The dramatic reduction of the required CPU time is primarily observed for the larger problem instances ($|S|$ = 80).

5.4. Pricing problem methodology

We analyse the different facets of the proposed pricing solution methodology in Section 5.4.1 and establish a benchmark with other pricing methodologies in Section 5.4.2.

5.4.1. Two-step pricing approach based on dynamic programming

In this section, we validate the performance of the different speed-up mechanisms embedded in the proposed two-step pricing procedure. First, we show the effectiveness of the devised pruning and symmetry breaking rules embedded in the dynamic programming algorithm. Second, we discuss the computational impact of introducing first a heuristic pricing step and its underlying concepts. In addition, we investigate the number of (diverse) columns that are returned for each student during a single pricing step. Note that, since the experiments in this section are time consuming, we report the results only for the smaller-sized instances ($|S| = 40$), without loss of generality.

5.4.1.1. Pruning and symmetry breaking rules. In order to show the added value of the pruning and symmetry breaking rules introduced in the dynamic programming algorithm to solve a single subproblem, we composed different versions of the proposed algorithm. Each version leaves out one specific speed-up technique leaving the rest of the algorithm unchanged. We exclude PR1 ('*w/o PR1*'), PR2 ('*w/o PR2*'), PR3 ('*w/o PR3*'), and the symmetry breaking rule ('*w/o SymB*'). We also considered a version that includes all speed-up techniques ('*w All*') and a version that leaves out all pruning rules and the symmetry breaking rule ('*w/o All*'). Table 5 presents the impact on the computational effort and displays the number of relevant nodes considered in the DP tree (#*Nodes*).

Table 5 shows that the removal of pruning rules and symmetry breaking deteriorates the performance of the algorithm as a result of a larger number of nodes visited and the larger computational time and memory required to store the information of the visited

Table 4
Computational impact of the lower bounds.

	w/o All	*Greedy - AN*	*Hungarian - AN*	*w All - RN*	*w All - AN*
$\#Itr_{DP}$	30	20	15	24	12
$\#Br_{DP}$	40	24	14	28	7
$Opt_{DP,RN}$	2%	3%	33%	35%	35%
$CPU_{DP,-}$	788	473	415	701	166
$Gap_{DP,-}$	0.2%	0.4%	0.1%	0.1%	0.0%
$Opt_{DP,-}$	93.3%	96.7%	96.7%	93.3%	100.0%

Table 5
The impact of pruning and symmetry breaking rules ($|S| = 40$)

	w/o All	*w/o PR1*	*w/o PR2*	*w/o PR3*	*w/o SymB*	*w All*
$\#Itr_{DP}$	8	12	12	12	12	10
$\#Br_{DP}$	2	6	6	2	6	4
$\#Nodes\ (\times 10^6)$	2.8	0.6	0.6	1.9	0.6	0.1
$CPU_{DP,PP}$	10155	225	186	1330	193	50
$CPU_{DP,-}$	10160	232	192	1337	199	54
$Gap_{DP,-}$	3%	0.0%	0.0%	0.2%	0.0%	0.0%
$Opt_{DP,-}$	80%	100.0%	100.0%	96.7%	100.0%	100.0%

nodes. This can be clearly discerned when omitting all pruning and symmetry breaking rules (*'w/o All'*). The largest impact is observed when PR3 is left out of the algorithm. Excluding PR3 (*'w/o PR3'*) increases the total number of visited nodes and the required computational effort significantly, i.e. the number of nodes ($\#Nodes$) increases from 0.1 to 1.9 million and the CPU time ($CPU_{DP,-}$) from 54 to 1337 s. In addition, for some instances the algorithm could not find the optimal solution ($Opt_{DP,T} = 96.7\%$).

5.4.1.2. First heuristic pricing step. In order to validate the two-step pricing approach, we analyse the impact of the heuristic first step to solve the pricing problem and investigate the validity of this step, the underlying principles and the number of student schedules retrieved in this step reflected by the solution pool size.

Validity of the heuristic pricing step - The first step to solve the pricing problem for a particular student is steered by the hospital capacity and truncates the dynamic programming algorithm according to a postulated time limit, identifying suitable columns in a heuristic manner. In order to evaluate the computational impact of these concepts, we evaluate different versions of the proposed algorithm in Table 6, i.e. the pricing procedure without the first heuristic pricing step (*'w/o First Step'*), without the time limit (*'w/o TLimit'*), without consideration of the hospital capacity constraints (*'w/o CLimit'*) and with both the time limit and capacity constraints included (*'w All'*).

Overall, Table 6 reveals that the removal of the time limit or the capacity constraints leads to a worse performance in terms of the computational effort ($CPU_{DP,-}$). Removing the postulated time limit (*'w/o TLimit'*) shows the largest impact on the required CPU time, which increases from 54s to 508s. Ignoring the hospital capacity constraints (*'w/o CLimit'*) increases the total number of column generation iterations $\#Itr_{DP}$ from 10 to 22 and the associated CPU time. This results from the fact that the consideration of the hospital capacity constraints supports the detection of (capacity-wise) consistent columns and a feasible solution for the restricted master problem. Leaving out the first heuristic step in solving the pricing problem (*'w/o First Step'*) has a detrimental effect on the computational performance of the procedure since only a single column is retrieved per student pricing problem. As a result, the number of column generation iterations ($\#Itr_{DP}$) and the associated CPU time to solve the master problem ($CPU_{DP,MP}$ = 211s) increase significantly. Consequently, not all instances are solved to optimality without the first heuristic pricing step.

Size of the solution pool - We analyse the performance of the proposed branch-and-price when varying the number of diverse columns retrieved in the first heuristic pricing step, which is controlled by the postulated solution pool size. In our procedure, we vary the solution pool size based upon the number of possible permutations related to the number disciplines, hospitals and time periods and a parameter ps to determine the solution pool size equal to $ps \cdot |D| \cdot |T| \cdot |H|$. In the experiment, the parameter ps is set to $0.005, 0.01, 0.02, 0.05$ and 0.1. In addition, we test the performance of the proposed algorithm when only a single column instead of multiple diverse columns is returned by the first step heuristic and, hence, the feature of adding a pool of columns to the master problem is omitted (*'w/o Solution pool'*). The results are displayed in Table 7.

Table 7 indicates that returning a pool of diverse columns leads to a better performance compared to returning only a single column per pricing problem (*'w/o Solution pool'*) in terms of the overall CPU time ($CPU_{DP,-}$) and, especially, of the time required for solving the pricing problems ($CPU_{DP,PP}$ = 343s). Further, to deter-

Table 6
The impact of the heuristic pricing step and underlying concepts ($|S| = 40$).

	w/o First Step	*w/o TLimit*	*w/o CLimit*	*w All*
$\#Itr_{DP}$	1135	10	22	10
$\#Br_{DP}$	1.4	4	8	4
$CPU_{DP,PP}$	453	503	268	50
$CPU_{DP,-}$	664	508	282	54
$Gap_{DP,-}$	2.7%	0.0%	0.0%	0.0%
$Opt_{DP,-}$	83.3%	100.0%	100.0%	100.0%

Table 7
Computational impact of solution pool size ($|S| = 40$).

	Solution pool size parameter (ps)					*w/o Solution pool*
	0.005	0.01	0.02	0.05	0.1	
$\#Itr_{DP}$	18	13	10	10	9	21
$\#Br_{DP}$	20	10	4	6	7	24
$\#ClC_{DP}$	772	834	1206	2324	3983	905
$CPU_{DP,PP}$	273	163	50	171	212	343
$CPU_{DP,MP}$	15	9	4	9	13	20
$CPU_{DP,-}$	288	172	54	180	225	363
$Gap_{DP,-}$	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
$Opt_{DP,-}$	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%

mine the ideal size of the solution pool, the table reveals a trade-off in the computational performance when varying the parameter ps. When the solution pool size is small ($ps = 0.005$ or 0.01), the total number of column generation iterations ($\#Itr_{DP}$) is high, which leads to large CPU times for solving both the restricted master and pricing problem. However, when the solution pool size is large ($ps = 0.05$ or 0.1), many columns need to be generated in the pricing step and a very large number of columns are considered in the restricted master problem ($\#ClC_{DP}$), which leads to again higher CPU times for solving the pricing and restricted master problem. The best setting is observed when ps = 0.02, which finds the best balance between the total number of branches, the total number of iterations and CPU times for solving the master and pricing problem.

5.4.2. Benchmark of the proposed pricing methodology

In this section, we benchmark the proposed pricing methodology with two other solution techniques that both rely on an acknowledged commercial solver, i.e.

(i) the MIP formulation of the pricing problem (see Section 4.1.2) is solved using the commercial solver CPLEX
(ii) a dedicated constraint programming formulation of the pricing problem (see Online Appendix D) is solved using the commercial solver ILOG-OPL

Table 8 summarises the results for different instance groups in order to denote the computational impact of the number of students ($|S|$), the discipline selection constrainedness for students ($R_{D,\kappa}$) and the defined type of precedence relationships ($PrRl$). The table shows that when the total number of students increases from 40 to 80, the required CPU time of the proposed procedure ($CPU_{DP,-}$) rises as a result of the increase in complexity of the problem under study. Similarly, CPU times ($CPU_{DP,-}$) are significantly higher for precedence relationship 2.1 and 1.2 as a result of the larger embedded complexity. The proposed procedure needs most CPU time for solving instances defining two predecessors in the precedence relationships ($PrRl$ = 2.1). These results are confirmed by the other pricing methodologies exploiting either a standard MIP solver or constraint programming. The impact of the discipline selection constrainedness for students, indicated by $R_{D,\kappa}$, is less obvious. When the standard MIP solver is used, the required CPU time ($CPU_{MIP,-}$) decreases when $R_{D,\kappa}$ is increased, i.e. the selection freedom decreases. When using constraint programming or dynamic programming, the CPU time ($CPU_{CP,-}$ and $CPU_{DP,-}$) increases when $|S| = 40$ whereas the CPU time decreases when $|S| = 80$.

When comparing the different solution methodologies, we observe that the proposed DP approach is able to solve all instances to optimality within the time limit within an acceptable CPU time ($CPU_{DP,-}$ = 166s). For all instance groups, the proposed methodology obtains the optimal solution for all instances with significantly less computational effort, outperforming the other solution methodologies. Using the standard MIP solver as pricing methodology increases the average CPU time ($CPU_{MIP,-}$) to 8054s. In that case, the procedure is not able to close the optimality gap within the time limit for 50% of the instances. When applying constraint programming, 88% of the instances are solved to optimality within

Table 8
Benchmarking the pricing solution methodology.

$\|S\|$	40						80						AVG
$R_{D,\kappa}$	0.6			0.75			0.6			0.75			
	1.1	1.2	2.1	1.1	1.2	2.1	1.1	1.2	2.1	1.1	1.2	2.1	
MIP													
$\#Br_{MIP}$	2	1	1	3	3	8	1	0	3	5	0	4	3
$CPU_{MIP,PP}$	2447	6459	3725	544	2821	1124	18918	20320	11087	8287	13956	6895	8048
$CPU_{MIP,-}$	2449	6462	3729	546	2824	1129	18923	20324	11094	8300	13960	6904	8054
$Gap_{MIP,-}$	18%	14%	10%	0%	5%	0%	3%	9%	6%	7%	20%	11%	9%
Opt_{MIP}	60%	40%	60%	100%	80%	100%	40%	20%	40%	20%	0%	40%	50%
CP													
$\#Br_{CP}$	3	3	2	5	4	4	4	5	8	7	7	6	5
$CPU_{CP,PP}$	40	173	121	56	370	211	4807	4682	3494	3143	12634	1754	2624
$CPU_{CP,-}$	44	177	124	60	374	215	4814	4692	3505	3150	12648	1762	2630
$Gap_{CP,-}$	0%	0%	0%	0%	0%	0%	0%	2%	0%	3%	4%	0%	1%
Opt_{CP}	100%	100%	100%	100%	100%	100%	80%	60%	100%	80%	40%	100%	88%
DP													
$\#Br_{DP}$	1	4	5	1	8	6	3	20	4	5	20	10	7
$CPU_{DP,PP}$	17	65	56	15	68	79	225	467	146	158	450	146	158
$CPU_{DP,-}$	19	69	61	18	75	85	231	486	152	166	472	158	166
$Gap_{DP,-}$	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Opt_{DP}	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%	100%

the time limit and the average CPU time ($CPU_{CP,-}$) amounts 2630s. The optimality gap ($Gap_{CP,-}$) is close to 1% and is larger than zero especially when $|S| = 80$.

5.5. General performance and benchmarking

In the following, we benchmark the general performance of the proposed branch-and-price algorithm with (i) a commercial MIP solver (Section 5.5.1) and (ii) a standard branch-and-price and show the improvements of the proposed algorithm by gradually introducing the different components (Section 5.5.2).

5.5.1. Proposed branch-and-price vs standard MIP solver

A first benchmark is attempted by applying the commercial solver CPLEX to the compact MIP model employing the original decision variables (see Section 3.2). However, the standard MIP solver fails to deliver a feasible solution with the time limit for instances with 40 and 80 students. In contrast, the proposed solution procedure solves all instances to optimality and requires 54s (277s) for instances with 40 (80) students, which shows the efficiency and the effectiveness of the proposed procedure. Note that the LP average upper bounds for the compact MIP model using original variables are equal to 3135 ($|S| = 40$) and 6483 ($|S| = 80$). In comparison, the decomposed formulation shows a tighter relaxation as the average LP upper bounds equal 3128 ($|S| = 40$) and 6469 ($|S| = 80$).

5.5.2. Proposed branch-and-price vs other B&P methodologies

In this section, we benchmark the proposed algorithm with a standard branch-and-price procedure, which conducts no dedicated lower bound calculations, uses a commercial MIP solver to solve the pricing problem, applies a best-first strategy to search the branching tree and considers only the branching rule on the original assignment variables (BR4). In addition, we gradually add the different components as proposed in our solution methodology to show the contribution of each component. Table 9 presents the performance of the following versions, i.e. (i) the standard branch-and-price procedure (‘*B&P* $(-|-|-|-|-)$’); (ii) with a basic pricing algorithm relying on dynamic programming (‘*B&P* $(DP|-|-|-|-)$’), which corresponds to the design choices of the procedure proposed by Beliën and Demeulemeester (2006); (iii) using the dedicated two-stage pricing methodology based on DP with application of dedicated pruning and symmetry breaking rules (‘*B&P* $(DP|2St|-|-|-)$’); (iv) incorporating the dedicated lower bound calculations (‘*B&P* $(DP|2St|LB|-|-)$’); (v) implementing the *right-first* search strategy (‘*B&P* $(DP|2St|LB|RF|-)$’); and (vi) with application of the mixed-shifting branching rule (‘*B&P* $(DP|2St|LB|RF|MS)$’), which is the proposed algorithm. Note that Table 9 also indicates the percentage of instances for which no feasible integer solution has been derived, indicated by $Inf_{DP,-}$.

Overall, Table 9 shows that the implemented dedicated mechanisms significantly improve the performance of the algorithm in terms of both the solution quality and the CPU time. The standard branch-and-price is able to solve only 33% of the instances to optimality ($Opt_{DP,-}$) within the postulated time limit. For the other 67% of the instances no feasible integer solution has been found. The average CPU time amounts 8619s. Using a standard dynamic programming approach to solve the pricing problem (‘*B&P* $(DP|-|-|-|-)$’), corresponding to Beliën and Demeulemeester (2006), the number of instances solved to optimality rises to 63% at the expense of a dramatic increase in CPU time. Implementing the proposed two-stage pricing algorithm with pruning rules (‘*B&P* $(DP|2St|-|-|-)$’) has a tremendous computational impact, decreasing the CPU time from 21458s to 4609s. Consequently, more instances can be solved to optimality within the time limit ($Opt_{DP,-}$ = 83%). Including lower bound calculations further improves the performance, resulting in a feasible solution for every instance. The number of instances solved to optimality is incremented to 95% while the required CPU time is significantly reduced to 238 s.

The results related to the composition of the branch-and-price tree and the selection of unexplored nodes reveal additional improvements. Implementing the right-first search strategy (‘*B&P* $(DP|2St|LB|RF|-)$’) increases the number of instances solved to optimality from 95% to 100% compared to the best-first search method. Using the mixed-shifting branching rule (‘*B&P* $(DP|2St|LB|RF|MS)$’) instead of applying solely *BR*4 reduces the number of created branches and the CPU time significantly. The CPU time decreases by almost 30% from 229s to 166s.

6. Conclusions

In this paper, we proposed a branch-and-price algorithm to solve the medical student scheduling problem that assigns students to medical disciplines and hospitals over the academic year, providing equitably-efficient solutions regarding to the student preferences. The proposed procedure is able to find optimal solutions within an acceptable time span outperforming standard optimisation approaches both in terms of computational effort and solution quality. The contribution of this paper is fourfold. First, we customised the search for an integer solution based on the optimal LP relaxation and devised a new search strategy to visit unexplored nodes and a new branching rule to create non-overlapping subproblems in the branching tree. We propose a right-first search strategy that selects the node with the largest number of right branching constraints imposed, which outperforms conventional search strategies such as best-first and depth-first. The applied mixed-shifting branching rule is a multi-layered branching rule that executes different branching principles in a specific order and outperforms the commonly applied binary branching rule on the original assignment variables. Second, we proposed a two-stage pricing algorithm that tries to find in a first heuristic stage a diverse set of columns, which reduces the number of column generation iterations significantly and, hence, has a positive effect

Table 9
Benchmark with different branch-and-price procedures.

	B&P $(-\|-\|-\|-\|-)$	*B&P* $(DP\|-\|-\|-\|-)$	*B&P* $(DP\|2St\|-\|-\|-)$	*B&P* $(DP\|2St\|LB\|-\|-)$	*B&P* $(DP\|2St\|LB\|RF\|-)$	*B&P* $(DP\|2St\|LB\|RF\|MS)$
$\#Itr_{DP}$	9	1793	44	13	14	12
$\#Br_{DP}$	5	3	33	10	12	7
$CPU_{DP,PP}$	8613	20585	4463	227	217	158
$CPU_{DP,-}$	8619	21458	4609	238	229	166
$Gap_{DP,-}$ (1)	0%	1%	0%	0%	0%	0%
$Inf_{DP,-}$	67%	25%	10%	0%	0%	0%
$Opt_{DP,-}$	33%	63%	83%	95%	100%	100%

(1) The value is calculated based on those instances the benchmark algorithm found a feasible solution.

on the tailing-off effect to prove LP optimality. Third, we developed different pruning and symmetry breaking rules to speed up the pricing step relying on dynamic programming and a benchmark is conducted with a standard MIP solver and constraint programming as pricing solution methodology. Fourth, we perform efficient lower bound calculations in every node to find high-quality integer solutions, reducing the size of the branching tree significantly.

CRediT authorship contribution statement

Babak Akbarzadeh: Conceptualization, Methodology, Software, Formal analysis, Data curation, Writing - original draft. **Broos Maenhout:** Conceptualization, Methodology, Investigation, Visualization, Writing - original draft, Writing - review & editing, Supervision, Project administration, Funding acquisition.

Appendix A. Supplementary data

Supplementary data associated with this article can be found, in the online version, athttps://doi.org/10.1016/j.cor.2021.105209.

References

Ahuja, R., Magnanti, T., Orlin, J., 1993. Network Flows: Theory, Algorithms, and Applications. Prentice Hall, New Jersey.

Akbarzadeh, B., Maenhout, B., 2020. A decomposition-based heuristic procedure for the Medical Student Scheduling problem. Eur. J. Oper. Res. 288 (1), 63–79.

Bard, J.F., Shu, Z., Leykum, L., 2013. Monthly clinic assignments for internal medicine housestaff. IIE Trans. Healthcare Syst. Eng. 3 (4), 207–239.

Bard, J.F., Shu, Z., Morrice, D.J., Leykum, L.K., 2016. Annual block scheduling for internal medicine residents with 4+1 templates. J. Oper. Res. Soc. 67 (7), 911–927.

Bard, J.F., Shu, Z., Morrice, D.J., Leykum, L.K., 2017. Constructing block schedules for internal medicine residents. IISE Trans. Healthcare Syst. Eng. 7 (1), 1–14.

Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance, P.H., 1998. Branch-and-price: Column generation for solving huge integer programs. Oper. Res. 46 (3), 316–329.

Beliën, J., Demeulemeester, E., 2006. Scheduling trainees at a hospital department using a branch-and-price approach. Eur. J. Oper. Res. 175 (1), 258–278.

Beliën, J., Demeulemeester, E., 2007. On the trade-off between staff-decomposed and activity-decomposed column generation for a staff scheduling problem. Ann. Oper. Res. 155 (1), 143–166.

Brech, C., Ernst, A., Kolisch, R., 2019. Scheduling medical residents' training at university hospitals. Eur. J. Oper. Res. 274 (1), 253–266.

Cohn, A., Root, S., Kymissis, C., Esses, J., Westmoreland, N., 2009. Scheduling medical residents at Boston University School of Medicine. Interfaces 39 (3), 186–195.

Colbert, C., French, J., Herring, M., 2017. Fairness: the hidden challenge for competency-based postgraduate medical education programs. Perspectives Med. Educ. 6 (6), 347–355.

Diponegoro, A., Rukman, F., 2018. Modeling of a Scheduling Method for Organizing Training Assignments. 104(Aisteel):184–187..

Doulabi, S.H.H., Rousseau, L.M., Pesant, G., 2016. A constraint-programming-based branch-and-price-and-cut approach for operating room planning and scheduling. INFORMS J. Comput. 28 (3), 432–448.

Franz, L.S., Miller, J.L., 1993. Scheduling Medical Residents to Rotations: Solving the Large-Scale Multiperiod Staff Assignment Problem. Oper. Res. 41 (2), 269–279.

Guo, J., Morrison, D.R., Jacobson, S.H., Jokela, J.A., 2014. Complexity results for the basic residency scheduling problem. J. Sched. 17 (3), 211–223.

Guo, M., Wu, S., Li, B., Song, J., Rong, Y., 2016. Integrated scheduling of elective surgeries and surgical nurses for operating room suites. Flexible Services Manuf. J. 28 (1–2), 166–181.

He, F., Qu, R., 2012. A constraint programming based column generation approach to nurse rostering problems. Computers Oper. Res. 39 (12), 3331–3343.

Hurink, J., Knust, S., 2001. List scheduling in a parallel machine environment with precedence constraints and setup times. Oper. Res. Lett. 5 (29), 231–239.

Ito, M., Onishi, A., Suzuki, A., Imamura, A., Ito, T., 2017. The resident scheduling problem: - A case study at Aichi Medical University Hospital. J. Jpn. Ind. Manage. Assoc. 68 (4E), 259–272.

Junker, U., Karisch, S.E., Kohl, N., Vaaben, B., Fahle, T., Sellmann, M., 1999. A framework for constraint programming based column generation. Lecture Notes Computer Sci. (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) 1713 (24), 261–275.

Karsu, O., Morton, A., 2015. Inequity averse optimization in operational research. Eur. J. Oper. Res. 2 (245), 343–359.

Kraul, S., Fügener, A., Brunner, J.O., Blobner, M., 2019. A robust framework for task-related resident scheduling. Eur. J. Oper. Res. 276 (2), 656–675.

Kuhn, H.W., 1955. The Hungarian method for the assignment problem. Naval Res. Logistics Quarterly 2 (1–2), 83–97.

Morrison, D.R., Jacobson, S.H., Sauppe, J.J., Sewell, E.C., 2016. Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning. Discrete Optim. 19, 79–102.

Nishi, T., Izuno, T., 2014. Column generation heuristics for ship routing and scheduling problems in crude oil transportation with split deliveries. Computers Chem. Eng. 60, 329–338.

Pisinger, D., Sigurd, M., 2007. Using decomposition techniques and constraint programming for solving the two-dimensional bin-packing problem. INFORMS J. Comput. 19 (1), 36–51.

Proano, R.A., Agarwal, A., 2018. Scheduling internal medicine resident rotations to ensure fairness and facilitate continuity of care. Health Care Manage. Sci. 21 (4), 461–474.

Rawls, J., 1971. A theory of justice. Harvard University Press, Cambridge, MA.

Ronnberg, E., Larsson, T., 2010. Automating the self-scheduling process of nurses in Swedish healthcare: a pilot study. Health Care Manage. Sci. 1 (13), 35–39.

Rose, A., Long, M.T., Hong, Y.-C., Perelstein, E., Cohn, A., 2015. Automation Improves Schedule Quality and Increases Scheduling Efficiency for Residents. J. Graduate Med. Educ. 8 (1), 45–49.

Ryan, K.A., Pruksaritanon, S., Reber, A.W., Aman, P.G., White, K.P., Smith, M.C., Hwang, S.N., 2013. Preference based scheduling for medical residents and interns at the University of Virginia. In: 2013 IEEE Systems and Information Engineering Design Symposium, SIEDS 2013, pp. 110–115.

Schleyer, T.K.L., 1994. Temple Dental School Uses an Expert System to Schedule Students' Clinical Rotations. Interfaces 24 (5), 13–21.

Smalley, H.K., Keskinocak, P., 2016. Automated medical resident rotation and shift scheduling to ensure quality resident education and patient care. Health Care Manage. Sci. 19 (1), 66–88.

Stolletz, R., Brunner, J.O., 2012. Fair optimization of fortnightly physician schedules with flexible shifts. Eur. J. Oper. Res. 219 (3), 622–629.

Trilling, L., Guinet, A., Le Magny, D., 2006. Nurse scheduling using integer linear programming and constraint programming. IFAC Proceedings Volumes (IFAC-PapersOnline) 12(PART, 1).